

End-to-End MLOps Pipeline

Cats vs Dogs Image Classifier for a Pet Adoption Platform

A production-grade lifecycle covering data versioning, experiment tracking, containerised inference, GitHub Actions CI/CD, Kubernetes deployment, post-deployment smoke tests, and live Prometheus-backed monitoring.

Course	MLOps (S1-25_AIMLCZG523)
Assignment	Assignment 2
Submission	Zip + Screen Recording + This Report
Video Demo	https://drive.google.com/drive/folders/1ZgW3GZscfUzRShQ7NYZ1Ruv_wSnAEIDe
Submitted by	2024AC05507 Kumar Sankalp
GitHub Link	GitHub - kumar-sankalp/mlops-assignment2

Table of Contents

1. Executive Summary
2. Project Analysis & Architecture
3. Repository Structure
4. Technology Stack
5. Module 1 — Model Development & Experiment Tracking
6. Module 2 — Model Packaging & Containerisation
7. Module 3 — Continuous Integration (CI)
8. Module 4 — Continuous Deployment (CD)
9. Module 5 — Monitoring, Logging & Live Performance Tracking
10. Post-Deployment Verification & Live Inference
11. Screenshots Index
12. Deliverables Checklist
13. How to Run the Project Locally

1. Executive Summary

This project implements a complete, production-ready MLOps lifecycle for a binary image classification use case (Cats vs Dogs) intended for a pet adoption platform. Every stage — from data acquisition and versioning to model training, containerised serving, CI/CD, Kubernetes deployment, and runtime monitoring — is automated and reproducible.

✓ What this submission demonstrates

A full lifecycle: Git + DVC for versioning, MLflow for experiment tracking, FastAPI for serving, Docker for packaging, GitHub Actions for CI/CD (with a self-hosted runner), Kubernetes/Minikube for deployment, and Prometheus + a custom Streamlit dashboard for live observability and drift detection.

Key Outcomes

Aspect	Implementation
Model	PyTorch SimpleCNN (3 Conv + 2 FC), input 3×224×224
Serialised size	~98 MB (model.pt)
Serving API	FastAPI with /health, /predict, /metrics
Container image	kumarsankalp/catdog on Docker Hub
Orchestration	Kubernetes (Minikube) — Deployment + Service
CI/CD	GitHub Actions — hosted CI + self-hosted CD runner
Observability	Prometheus /metrics + Streamlit monitoring tab
Drift detection	Batch-level accuracy check with labelled assets

2. Project Analysis & Architecture

The pipeline is split into three distinct environments that mirror how real-world ML systems are operated: a Local Development environment for training and experimentation, a GitHub-hosted CI/CD environment for automated build and test, and a Kubernetes-based Serving environment for production traffic.

2.1 Architectural Overview

The end-to-end flow, top-to-bottom:

- **Data Acquisition & Versioning:** Kaggle → DVC-tracked raw data → preprocessing (224×224, augmentation).
- **Model Development:** Jupyter notebook + train.py, tracked in MLflow (params, metrics, model artifact).
- **Packaging:** FastAPI + torchvision preprocessing wrapped in a slim python:3.9-slim Docker image.
- **Continuous Integration:** GitHub Actions on ubuntu-latest — pytest, docker build, push to Docker Hub with the Git SHA tag.
- **Continuous Deployment:** Self-hosted runner rewrites the image tag in the K8s manifests, applies them to Minikube, and rolls the deployment.
- **Verification:** Post-deploy smoke test hits /health and /predict via kubectl port-forward.
- **Monitoring & Drift:** Prometheus /metrics scraped by the Streamlit dashboard; batch benchmark reports live accuracy.

Design decisions worth calling out

- Two requirements files (mac vs Linux) isolate torch/torchvision wheel conflicts between development and production.
- The self-hosted CD runner sidesteps the fact that GitHub-hosted runners cannot reach a local Minikube cluster.
- Kubernetes port-forwarding is used in the smoke-test job because macOS Docker-driver Minikube does not expose a routable LoadBalancer IP.
- Prometheus metrics are surfaced directly in the Streamlit UI to keep the entire observability stack in one place for the demo.

3. Repository Structure

The project is organised for clarity of responsibility between data/model code, the serving app, infrastructure manifests, and CI/CD.

Directory tree

```
MLOPS2_Submission/
├── .github/workflows/ci-cd.yml    # GitHub Actions CI + CD pipeline
├── .dvc/                         # DVC configuration (data versioning)
├── app/
│   ├── main.py                  # FastAPI inference service
│   ├── ui.py                    # Streamlit UI + monitoring dashboard
│   ├── assets/                  # Labelled cat/dog images for benchmark
│   └── tests/                   # pytest unit + integration tests
├── src/
│   ├── model.py                 # SimpleCNN architecture (PyTorch)
│   ├── data_preprocessing.py     # Dataset, transforms, dataloaders
│   ├── train.py                 # Training loop + MLflow logging
│   └── download_data.py         # Kaggle download helper
├── k8s/                         # Deployment + Service manifests
├── scripts/
│   ├── deploy_local.sh          # One-shot Minikube deploy
│   ├── smoke_test.py            # Post-deploy health + predict check
│   ├── performance_tracker.py   # Batch accuracy tracker
│   └── update_docx.py           # (this file) doc generator
├── notebooks/mlops_pipeline.ipynb # EDA + training walk-through
├── Dockerfile                   # Container definition
├── docker-compose.yml           # Local multi-service run
├── requirements.txt              # Pinned deps for Linux/Docker
├── requirements-mac.txt         # Pinned deps for macOS dev
├── model.pt                     # Trained PyTorch weights
├── mlflow.db                    # MLflow SQLite backend
└── docs/                        # PDF spec + this report
```

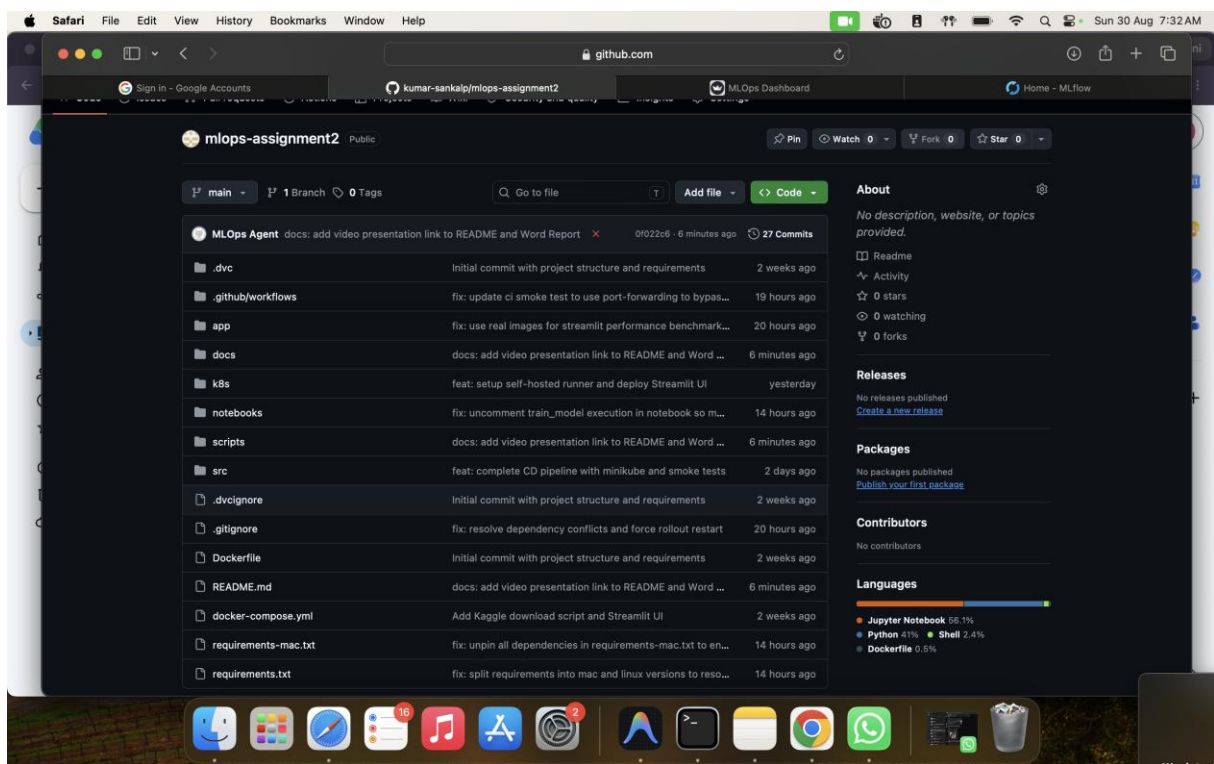


Figure — Repository layout as browsed on GitHub — every module of the pipeline is versioned together.

4. Technology Stack

Layer	Tooling	Purpose
Data versioning	Git + DVC	Source and dataset lineage
Modelling	PyTorch, torchvision	SimpleCNN + preprocessing
Experiment tracking	MLflow (SQLite backend)	Runs, params, metrics, artifacts
Serving	FastAPI + Uvicorn	/health, /predict, /metrics
UI	Streamlit	Inference + monitoring dashboard
Packaging	Docker (python:3.9-slim)	Reproducible container image
Registry	Docker Hub	Image storage & distribution
CI/CD	GitHub Actions (hosted + self-hosted)	Test, build, push, deploy
Orchestration	Kubernetes (Minikube)	Deployments, Services, rollouts
Monitoring	Prometheus FastAPI Instrumentator	Request count, latency, memory
Post-deploy checks	pytest + custom smoke_test.py	Health + prediction verification

5. Module 1 — Model Development & Experiment Tracking

Objective: Build a baseline model, track experiments, and version all artifacts. This module produces the trained `model.pt` that every other stage of the pipeline consumes.

5.1 Data Versioning with DVC

The Kaggle dataset is downloaded once and tracked with DVC so that the exact pre-processed data used for training is reproducible on any machine without re-committing large binary files to Git.

5.2 Model Architecture (SimpleCNN)

A compact 3-block Convolutional Neural Network with a fully-connected head. The model outputs raw logits and is trained with `BCEWithLogitsLoss` for numerical stability.

src/model.py — PyTorch model

```
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # Input size: 3 x 224 x 224
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)

        self.pool = nn.MaxPool2d(2, 2)

        # After 3 pooling layers: 224 -> 112 -> 56 -> 28
        self.fc1 = nn.Linear(64 * 28 * 28, 512)
        self.fc2 = nn.Linear(512, 1) # Binary classification

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))

        x = x.view(-1, 64 * 28 * 28)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        # Using BCEWithLogitsLoss during training, so no sigmoid here.
        return x
```

5.3 Preprocessing & Data Split

Images are resized to 224×224, normalized using ImageNet statistics, and augmented (random horizontal flip + random rotation) on the training set only. The dataset is split 80/10/10 into train/val/test with stratification.

src/data_preprocessing.py (excerpt)

```
import os
import glob
```



```

from PIL import Image
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from sklearn.model_selection import train_test_split
import numpy as np

class CatsDogsDataset(Dataset):
    def __init__(self, file_paths, labels, transform=None):
        self.file_paths = file_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.file_paths)

    def __getitem__(self, idx):
        img_path = self.file_paths[idx]
        image = Image.open(img_path).convert('RGB')
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, torch.tensor([label], dtype=torch.float32)

def generate_dummy_data(raw_data_dir, num_samples=100):
    """Generates dummy data for testing if real data is not found."""
    print(f"Generating {num_samples} dummy images in {raw_data_dir}...")
    cats_dir = os.path.join(raw_data_dir, 'Cat')
    dogs_dir = os.path.join(raw_data_dir, 'Dog')
    os.makedirs(cats_dir, exist_ok=True)
    os.makedirs(dogs_dir, exist_ok=True)

    for i in range(num_samples // 2):
        # Create a random RGB image for Cat
        img = Image.fromarray(np.random.randint(0, 255, (224, 224, 3), dtype=np.uint8))
        img.save(os.path.join(cats_dir, f"{i}.jpg"))

        # Create a random RGB image for Dog
        img = Image.fromarray(np.random.randint(0, 255, (224, 224, 3), dtype=np.uint8))
        img.save(os.path.join(dogs_dir, f"{i}.jpg"))

def get_dataloaders(raw_data_dir='data/raw', batch_size=32):
    cat_files = glob.glob(os.path.join(raw_data_dir, 'Cat', '*.jpg'))
    dog_files = glob.glob(os.path.join(raw_data_dir, 'Dog', '*.jpg'))

    if len(cat_files) == 0 or len(dog_files) == 0:
        print("Real dataset not found. Generating dummy dataset for testing purposes.")
        generate_dummy_data(raw_data_dir)
        cat_files = glob.glob(os.path.join(raw_data_dir, 'Cat', '*.jpg'))
        dog_files = glob.glob(os.path.join(raw_data_dir, 'Dog', '*.jpg'))

    all_files = cat_files + dog_files
    # 0 for cat, 1 for dog
    labels = [0]*len(cat_files) + [1]*len(dog_files)

```

```
# Check for valid images
```

5.4 Training with MLflow Tracking

The training loop logs hyperparameters, per-epoch train/validation loss, validation accuracy, and the resulting model.pt as an artifact to MLflow.

src/train.py (excerpt)

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
import mlflow
import mlflow.pytorch
from model import SimpleCNN
from data_preprocessing import get_dataloaders

def train_model():
    # Hyperparameters
    batch_size = 32
    learning_rate = 0.001
    num_epochs = 5

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Load data
    train_loader, val_loader, _ = get_dataloaders(batch_size=batch_size)

    # Initialize model, loss function, and optimizer
    model = SimpleCNN().to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    # Start MLflow run
    mlflow.set_experiment("Cats_vs_Dogs_Classification")
    with mlflow.start_run():
        # Log parameters
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("learning_rate", learning_rate)
        mlflow.log_param("num_epochs", num_epochs)
        mlflow.log_param("optimizer", "Adam")

        for epoch in range(num_epochs):
            model.train()
            running_loss = 0.0

            for inputs, labels in train_loader:
                inputs, labels = inputs.to(device), labels.to(device)

                optimizer.zero_grad()

                outputs = model(inputs)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
```

```

        running_loss += loss.item() * inputs.size(0)

    epoch_loss = running_loss / len(train_loader.dataset)
    print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {epoch_loss:.4f}")
    mlflow.log_metric("train_loss", epoch_loss, step=epoch)

# Validation

```

The screenshot shows a Jupyter notebook interface with a file explorer on the left and a code editor in the center. The code editor displays the following Python code:

```

import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)

        self.fc1 = nn.Linear(64 * 28 * 28, 512)
        self.fc2 = nn.Linear(512, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = x.view(-1, 64 * 28 * 28)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN()
print(model)

[32] ✓ 0.1s

SimpleCNN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=50176, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=1, bias=True)
)

```

The bottom of the notebook shows the output of the training process, indicating that all smoke tests passed successfully.

Figure — Jupyter notebook — SimpleCNN model summary and training output during EDA.

```
!pip install -r ../requirements-mac.txt
[29] ✓ 2.8s Python

Requirement already satisfied: fastapi in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 1))
Requirement already satisfied: uvicorn in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 2))
Requirement already satisfied: torch in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 3))
Requirement already satisfied: torchvision in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 4))
Requirement already satisfied: mlflow in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 5))
Requirement already satisfied: dvc in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 6))
Requirement already satisfied: pytest in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 7))
Requirement already satisfied: requests in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 8))
Requirement already satisfied: Pillow in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 9))
Requirement already satisfied: numpy in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 10))
Requirement already satisfied: scikit-learn in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 11))
Requirement already satisfied: kaggle in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 12))
Requirement already satisfied: streamlit in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 13))
Requirement already satisfied: python-multipart in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from -r ../requirements-mac.txt (line 14))
Collecting httpx (from -r ../requirements-mac.txt (line 15))
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting prometheus-fastapi-instrumentator (from -r ../requirements-mac.txt (line 16))
  Downloading prometheus_fastapi_instrumentator-8.1.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: starlette<=0.46.0 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
Requirement already satisfied: pydantic<=2.9.0 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
Requirement already satisfied: typing-extensions<=4.8.0 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
Requirement already satisfied: typing-inspection<=0.4.2 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
Requirement already satisfied: annotated-doc<=0.0.2 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
Requirement already satisfied: click<=7.0 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
Requirement already satisfied: h11<=0.8 in /Users/kumarsankalp/Downloads/MLOPS2/venv/lib/python3.14/site-packages (from prometheus-fastapi-instrumentator)
...
Successfully installed httpcore-1.0.9 httpx-0.28.1 prometheus-client-0.26.0 prometheus-fastapi-instrumentator-8.1.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Figure — Jupyter notebook — dependency installation and environment bootstrap.

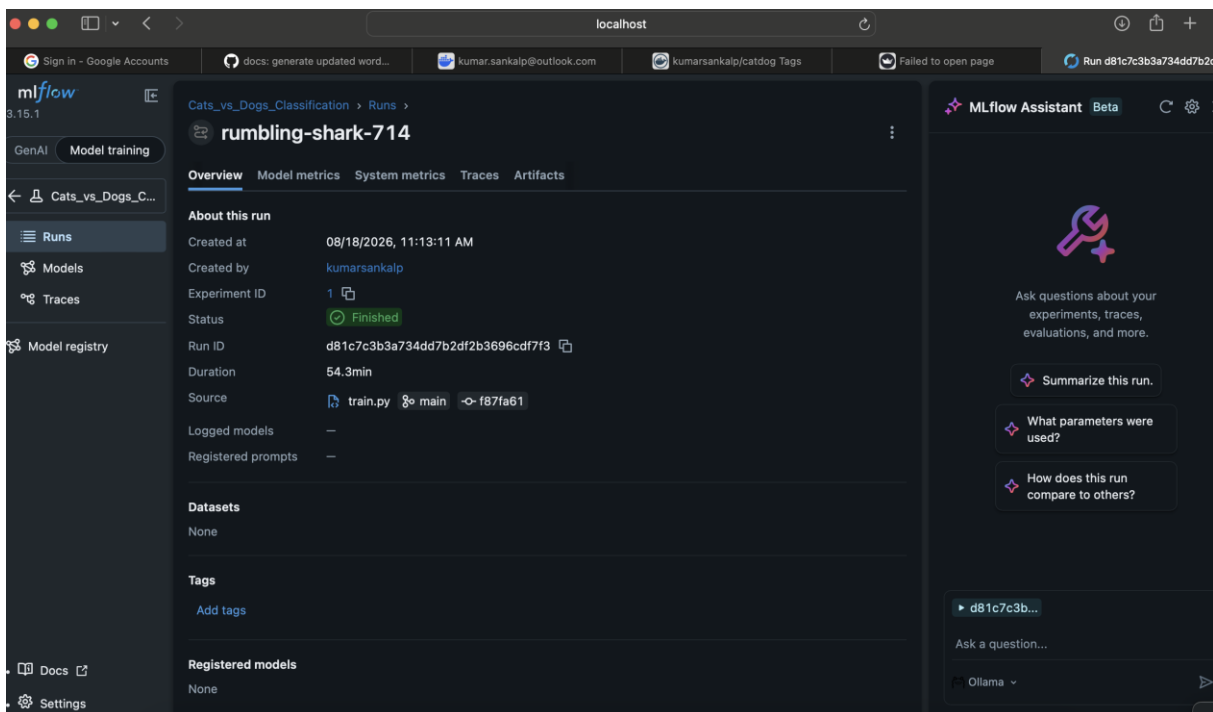


Figure — MLflow — logged training run with parameters, metrics, and artifact tree.

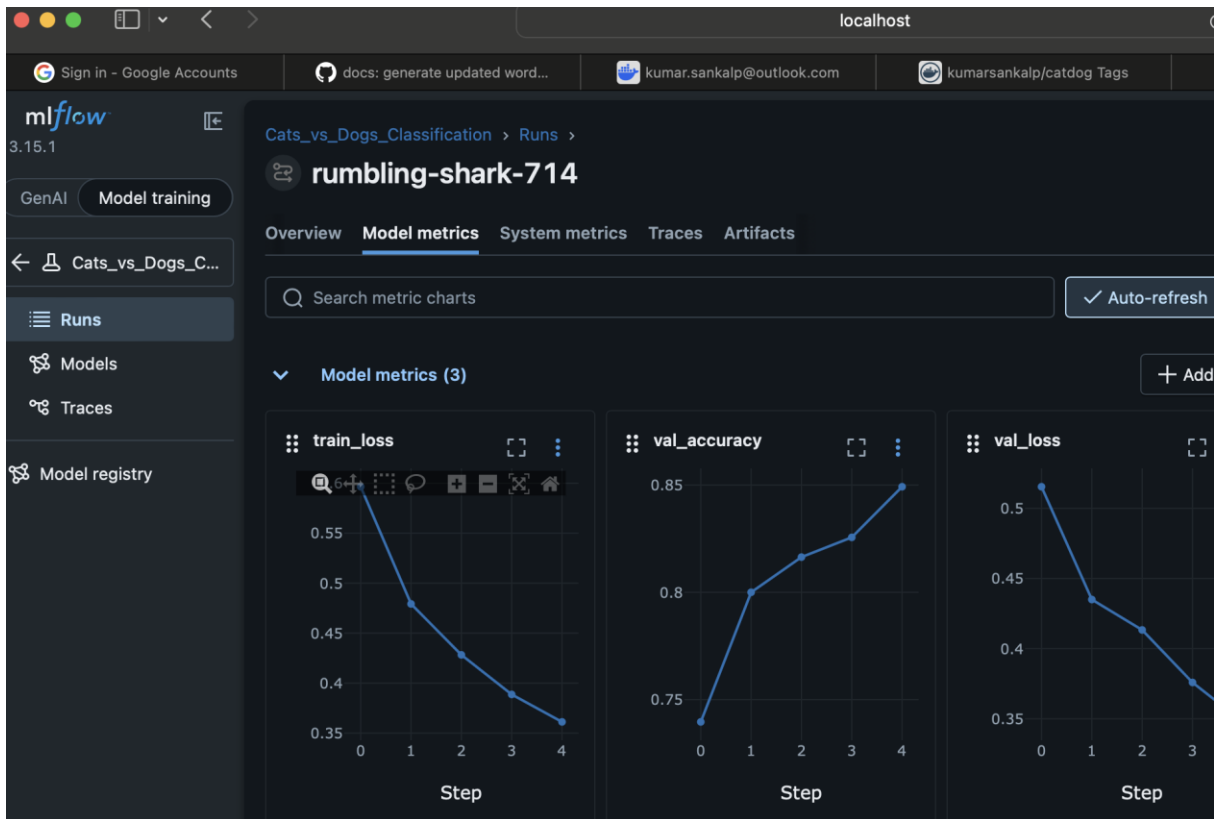


Figure — MLflow — per-epoch validation loss and accuracy plotted over the run.

💡 Why MLflow, not just print logs?

MLflow captures the full experimental record — parameters, metric curves, and the trained artifact — into a queryable store. That record is what makes the downstream pipeline reproducible: any commit → any container → the same model.

6. Module 2 — Model Packaging & Containerisation

Objective: Wrap the trained model in a reproducible container that exposes a REST inference API.

6.1 FastAPI Inference Service

The service exposes three routes:

Endpoint	Method	Purpose
/	GET	Redirect to interactive Swagger docs
/health	GET	Liveness / readiness probe for Kubernetes
/predict	POST	Accepts an image upload, returns label + probability
/metrics	GET	Prometheus scrape target (auto-generated)

app/main.py — FastAPI service (excerpt)

```
from fastapi import FastAPI, UploadFile, File, HTTPException
import torch
import torchvision.transforms as transforms
from PIL import Image
import io
import sys
import os

# Add src to Python path so we can import model
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'src')))
try:
    from model import SimpleCNN
except ImportError:
    # Handle the case where src is not accessible during some tests
    class SimpleCNN(torch.nn.Module):
        def __init__(self):
            super().__init__()
            self.fc = torch.nn.Linear(3*224*224, 1)
        def forward(self, x):
            x = x.view(-1, 3*224*224)
            return self.fc(x)

from prometheus_fastapi_instrumentator import Instrumentator

app = FastAPI(title="Cats vs Dogs Classification API", version="1.0.0")

# Instrument the app for Prometheus metrics (request count, latency)
Instrumentator().instrument(app).expose(app)

# Setup model
device = torch.device('cpu')
model = SimpleCNN()
```

```

# Path to trained model
MODEL_PATH = os.path.join(os.path.dirname(__file__), '..', 'model.pt')

try:
    if os.path.exists(MODEL_PATH):
        model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
        print("Model loaded successfully")
    else:
        print(f"Warning: Model file not found at {MODEL_PATH}. Using untrained model.")
except Exception as e:
    print(f"Warning: Could not load model: {e}")

model.eval()

# Preprocessing transform
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

from fastapi.responses import RedirectResponse

```

6.2 Pinned Dependencies for Reproducibility

All ML libraries are strictly version-pinned to avoid the classic torchvision/Pillow/streamlit dependency-resolver deadlock.

requirements.txt

```

fastapi==0.103.1
uvicorn==0.23.2
torch==2.0.1
torchvision==0.15.2
mlflow==2.6.0
dvc==3.15.0
pytest==7.4.2
requests==2.31.0
Pillow==9.5.0
numpy==1.24.3
scikit-learn==1.3.0
kaggle==1.5.16
streamlit==1.25.0
python-multipart==0.0.6
httpx==0.24.1
prometheus-fastapi-instrumentator==6.1.0

```

6.3 Dockerfile

A lean image built from python:3.9-slim. Source, model weights and app are copied in explicit layers so that only the source changes bust the pip cache.

Dockerfile

```

FROM python:3.9-slim

WORKDIR /app

```

```
# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy source code and application code
# We copy src as well because app/main.py depends on src/model.py
COPY src/ /app/src/
COPY app/ /app/app/

# Copy model artifact
# In a real setup, we might download this from a model registry
COPY model.pt /app/model.pt

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

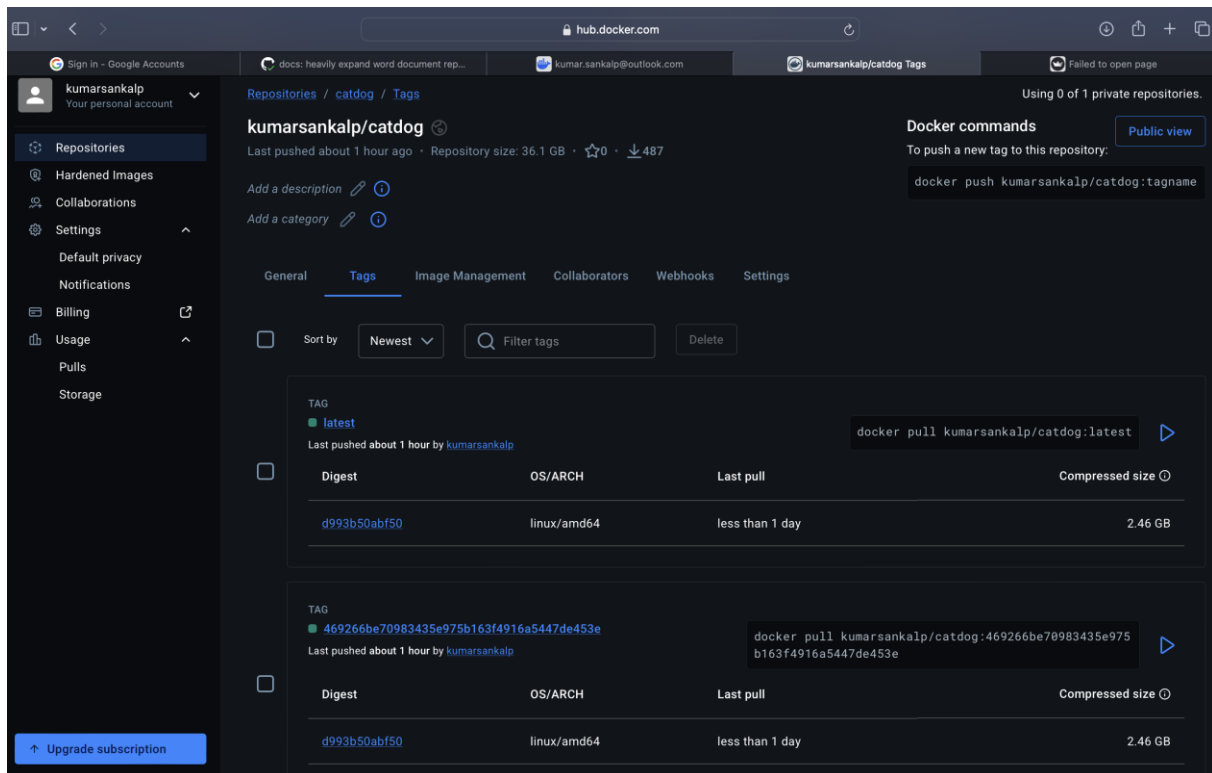


Figure — Docker Hub — image repository showing tagged releases pushed by CI.

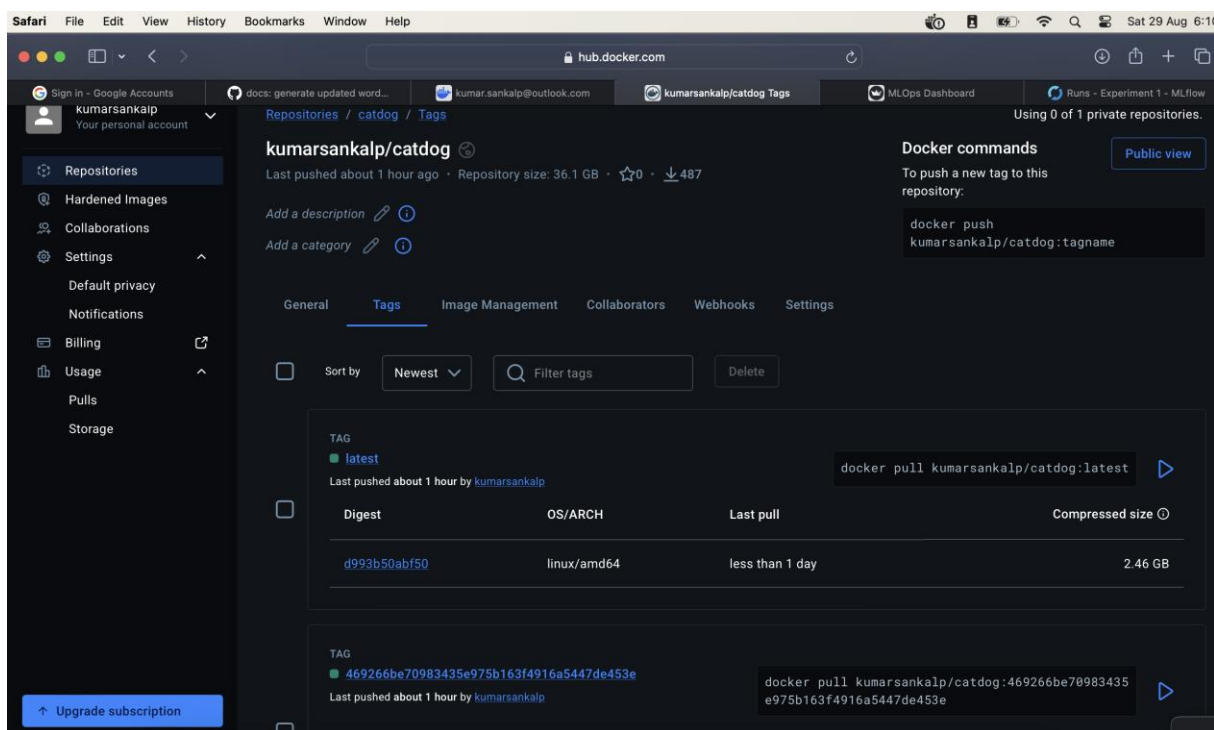


Figure — Docker Hub — image tagged with the exact Git SHA of the commit that built it.

✓ Immutable tags = deployable history

Because every CI run tags the image with the commit SHA, a rollback is one `kubectl set image` away, and every running pod can be traced back to the exact line of source code that produced it.

7. Module 3 — Continuous Integration (CI)

Objective: Every push and pull request to main is automatically tested, packaged, and published to a container registry.

7.1 Pipeline Definition

.github/workflows/ci-cd.yml — build-and-push job

```
name: MLOps CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Python 3.9
        uses: actions/setup-python@v4
        with:
          python-version: 3.9

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run Pytest
        run: |
          PYTHONPATH=src pytest app/tests/

      - name: Build Docker image
        if: success()
        run: |
          # Using a dummy model file for the docker build if not using DVC in CI
          touch model.pt
          docker build -t mlops-inference:latest .

      - name: Log in to Docker Hub
        if: github.event_name == 'push' && github.ref == 'refs/heads/main'
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Push Docker image
        if: github.event_name == 'push' && github.ref == 'refs/heads/main'
        run: |
```

```

docker tag mlops-inference:latest ${ secrets.DOCKERHUB_USERNAME }}/catdog:${{
github.sha }}
docker tag mlops-inference:latest ${ secrets.DOCKERHUB_USERNAME }}/catdog:latest
docker push ${ secrets.DOCKERHUB_USERNAME }}/catdog:${{ github.sha }}
docker push ${ secrets.DOCKERHUB_USERNAME }}/catdog:latest

```

7.2 Automated Tests

The pytest suite covers a preprocessing utility and the two most important API contracts (health check + prediction shape).

app/tests/test_api.py

```

import os
import sys
from fastapi.testclient import TestClient
from PIL import Image
import io

# Add app to Python path
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from main import app

client = TestClient(app)

def test_health_check():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "healthy"}

def test_predict_endpoint_valid_image():
    # Create a dummy image
    img = Image.new('RGB', (224, 224), color = 'red')
    img_byte_arr = io.BytesIO()
    img.save(img_byte_arr, format='JPEG')
    img_byte_arr = img_byte_arr.getvalue()

    files = {'file': ('test.jpg', img_byte_arr, 'image/jpeg')}
    response = client.post("/predict", files=files)

    assert response.status_code == 200
    json_data = response.json()
    assert "prediction" in json_data
    assert "probability" in json_data
    assert json_data["prediction"] in ["Cat", "Dog"]

def test_predict_endpoint_invalid_file():
    # Provide a text file instead of an image
    files = {'file': ('test.txt', b"Hello World", 'text/plain')}
    response = client.post("/predict", files=files)

    assert response.status_code == 400
    assert "not an image" in response.json()["detail"]

```

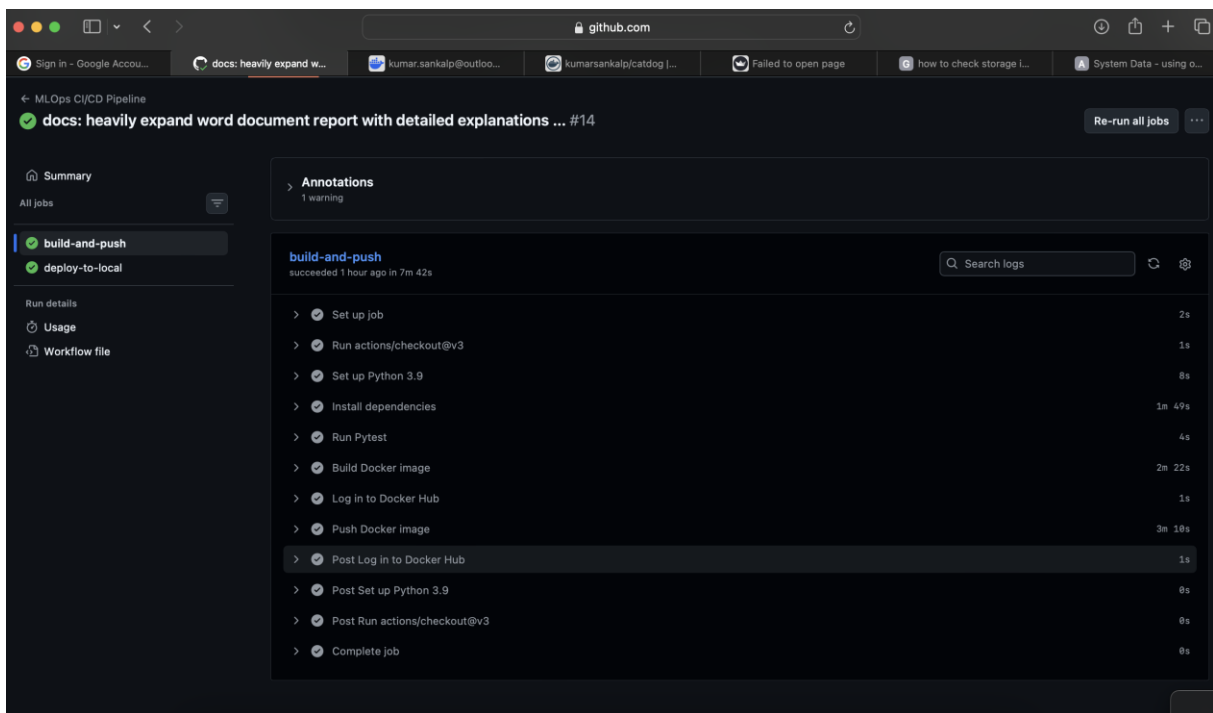


Figure — GitHub Actions — pipeline overview with the CI and CD jobs in sequence.

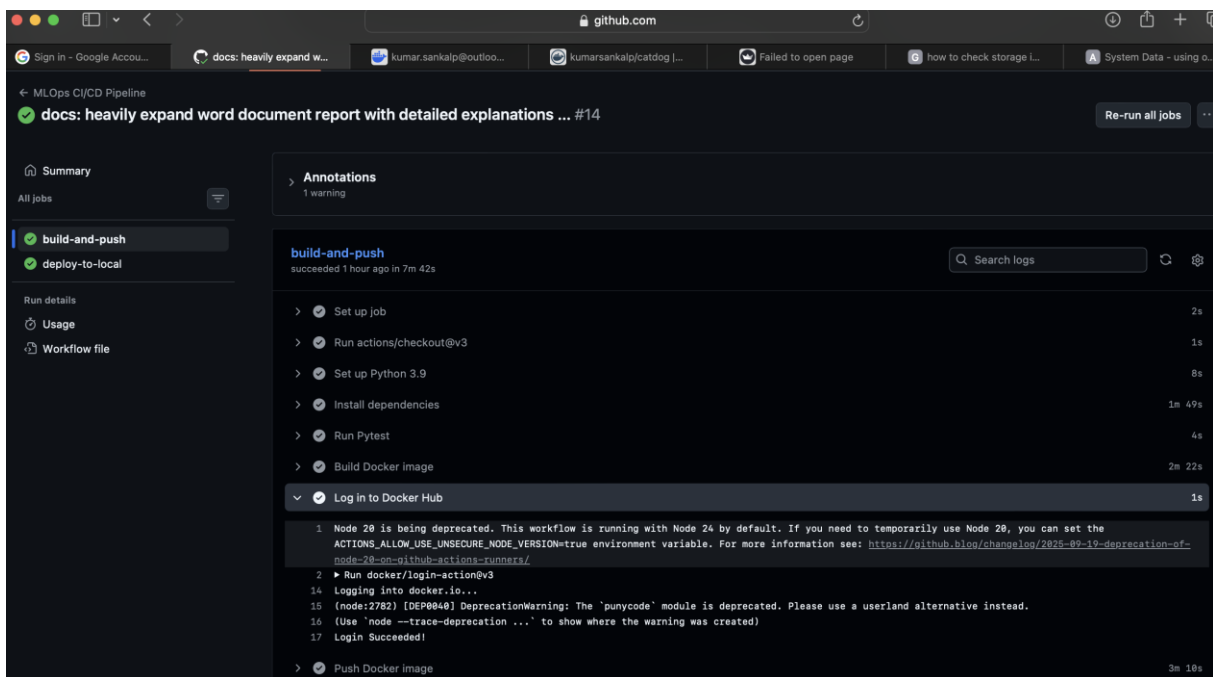


Figure — GitHub Actions — CI job (build-and-push) running on ubuntu-latest.

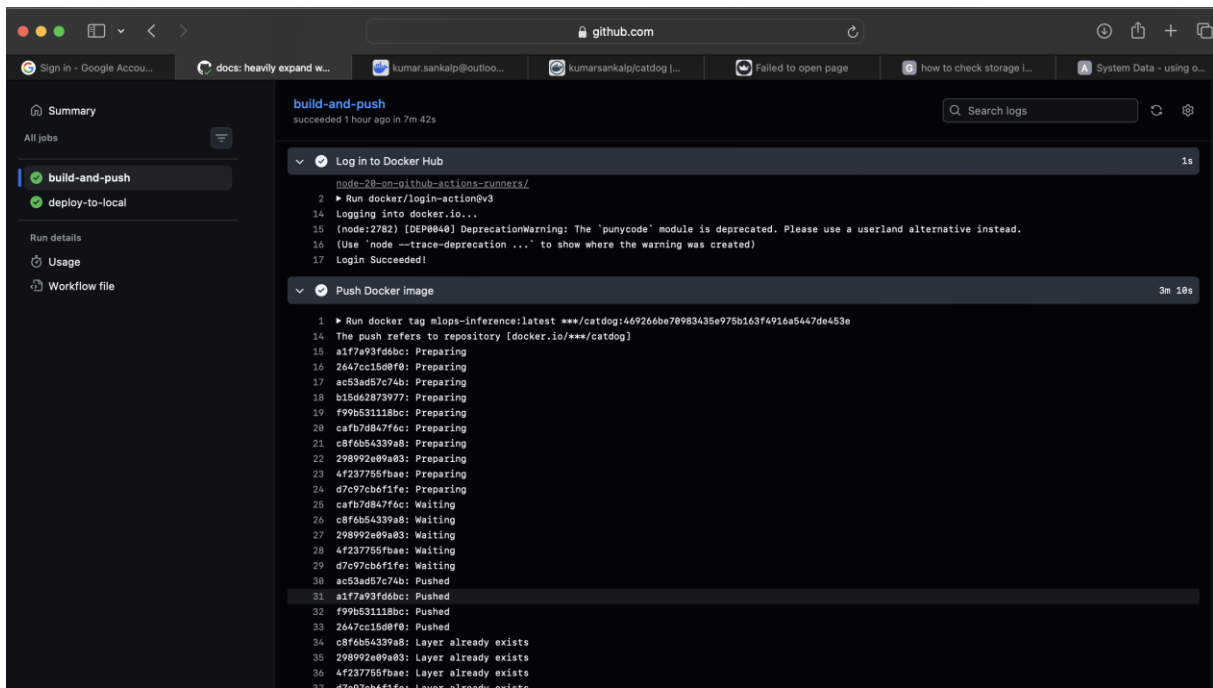


Figure — GitHub Actions — detailed CI step logs (pytest → docker build → push).

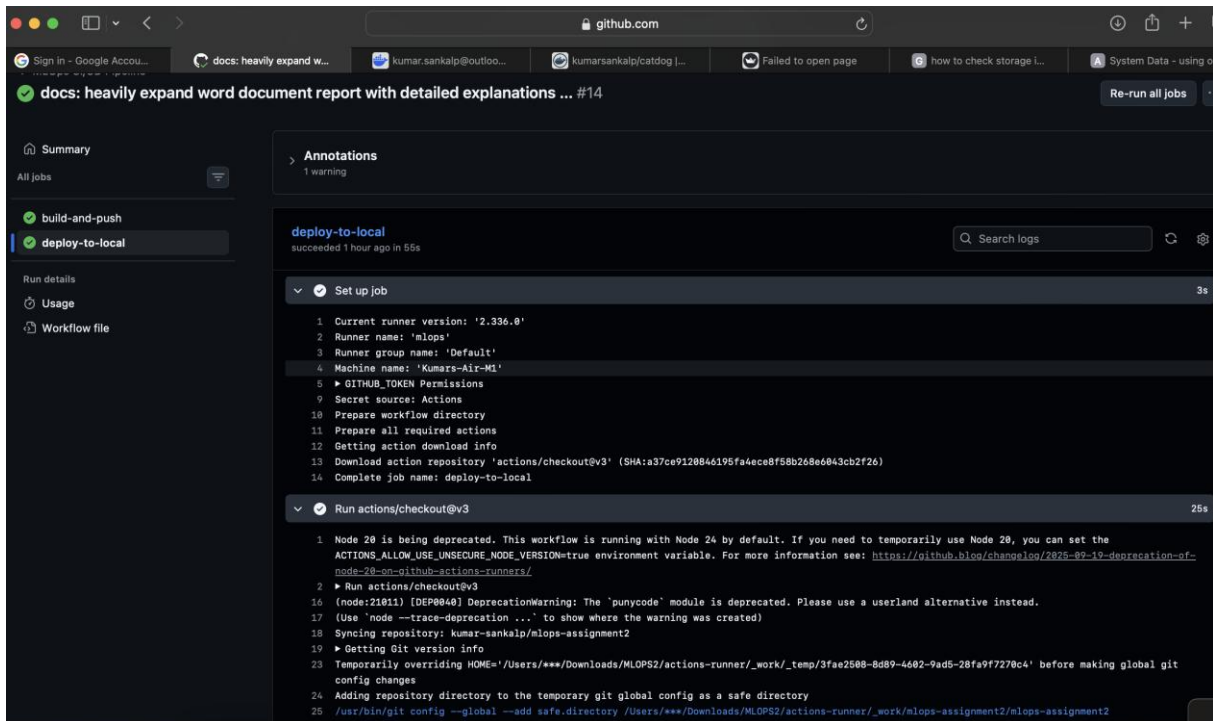


Figure — GitHub Actions — CI stage that hands off to the self-hosted runner.

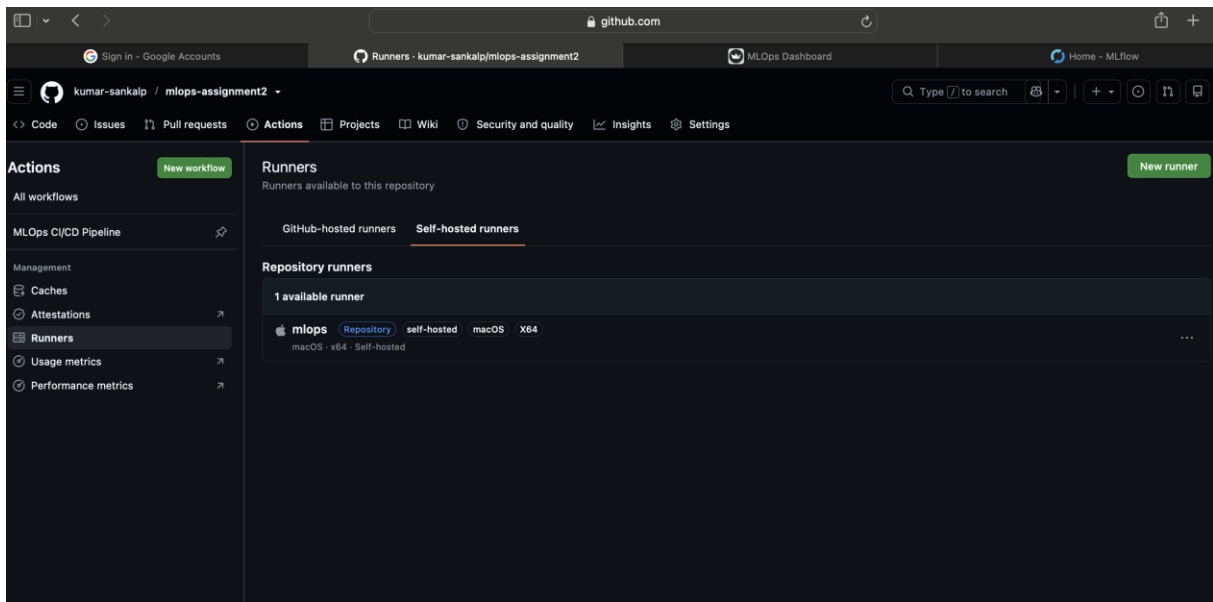


Figure — Self-hosted runner registered and picking up the workflow job.

8. Module 4 — Continuous Deployment (CD)

Objective: The image published in CI is automatically rolled out to a Kubernetes cluster, and a post-deploy smoke test verifies the service.

8.1 Kubernetes Manifests

k8s/deployment.yaml — FastAPI backend Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mlops-inference
  labels:
    app: mlops-inference
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mlops-inference
  template:
    metadata:
      labels:
        app: mlops-inference
    spec:
      containers:
        - name: inference-api
          image: kumarsankalp/catdog:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          livenessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 15
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /health
              port: 8000
            initialDelaySeconds: 5
            periodSeconds: 5
```

k8s/streamlit-deployment.yaml — Streamlit UI Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: streamlit-ui
  labels:
    app: streamlit-ui
spec:
  replicas: 1
  selector:
```

```

matchLabels:
  app: streamlit-ui
template:
  metadata:
    labels:
      app: streamlit-ui
  spec:
    containers:
      - name: streamlit-ui
        image: kumarsankalp/catdog:latest
        imagePullPolicy: IfNotPresent
        command: ["streamlit", "run", "app/ui.py", "--server.port", "8501", "--server.address", "0.0.0.0"]
        env:
          - name: API_URL
            value: "http://mlops-inference-service:8000"
        ports:
          - containerPort: 8501

```

8.2 Local Deploy Script

scripts/deploy_local.sh (excerpt)

```

#!/bin/bash
set -e

# Always execute from the repository root
cd "$(dirname "$0")/.."

echo "Deploying to local Minikube cluster..."

# Ensure minikube is running
if ! minikube status > /dev/null 2>&1; then
  echo "Starting Minikube..."
  minikube start
fi

# Point docker to minikube's docker daemon
eval $(minikube docker-env)

# Build the image locally inside minikube
echo "Building Docker image..."
docker build -t kumarsankalp/catdog:latest .

# Apply K8s manifests
echo "Applying Kubernetes manifests..."
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/streamlit-deployment.yaml
kubectl apply -f k8s/streamlit-service.yaml

# Force pods to restart to pick up the newly built Docker image
echo "Restarting deployments to pick up new image..."
kubectl rollout restart deployment/mlops-inference
kubectl rollout restart deployment/streamlit-ui

# Wait for rollout

```



```

echo "Waiting for deployments to be ready..."
kubectl wait --for=condition=available --timeout=120s deployment/mlops-inference
kubectl wait --for=condition=available --timeout=120s deployment/streamlit-ui

# Setup Python Virtual Environment for Smoke Test locally
echo "Setting up virtual environment for smoke tests..."

```

8.3 Smoke Test

scripts/smoke_test.py

```

import requests
import sys
import time

def run_smoke_test(base_url="http://localhost:8000"):
    print(f"Running smoke tests against {base_url}...")

    # Wait for service to be up
    max_retries = 5
    for i in range(max_retries):
        try:
            response = requests.get(f"{base_url}/health")
            if response.status_code == 200:
                print("Health check passed.")
                break
        except requests.exceptions.ConnectionError:
            if i == max_retries - 1:
                print("Failed to connect to the service.")
                sys.exit(1)
            print("Service not ready, retrying in 5 seconds...")
            time.sleep(5)

    # Test Prediction (Needs a dummy image)
    from PIL import Image
    import io

    print("Testing prediction endpoint...")
    img = Image.new('RGB', (224, 224), color = 'blue')
    img_byte_arr = io.BytesIO()
    img.save(img_byte_arr, format='JPEG')
    img_byte_arr = img_byte_arr.getvalue()

    files = {'file': ('dummy.jpg', img_byte_arr, 'image/jpeg')}
    try:
        response = requests.post(f"{base_url}/predict", files=files)
        if response.status_code == 200:
            print(f"Prediction successful: {response.json()}")
        else:
            print(f"Prediction failed with status {response.status_code}: {response.text}")
            sys.exit(1)
    except Exception as e:
        print(f"Prediction request failed: {e}")
        sys.exit(1)

    print("All smoke tests passed successfully!")

```

```
if __name__ == "__main__":
    url = sys.argv[1] if len(sys.argv) > 1 else "http://localhost:8000"
    run_smoke_test(url)
```

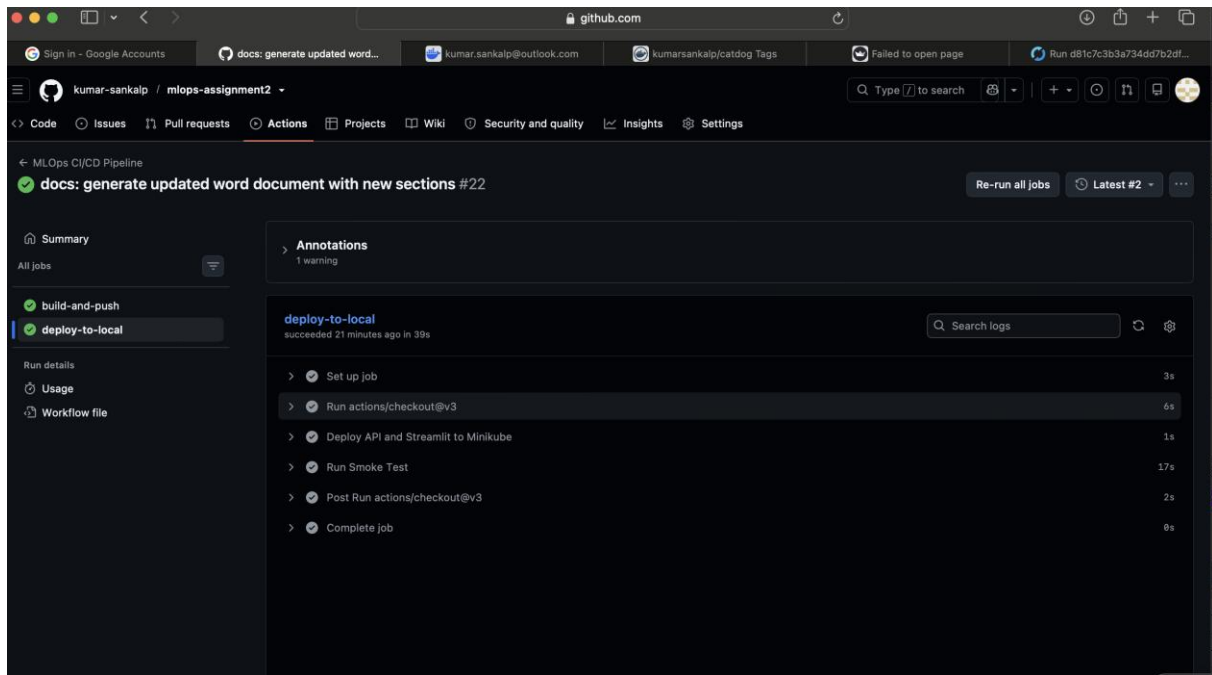


Figure — GitHub Actions — CD job overview (deploy-to-local depends on build-and-push).

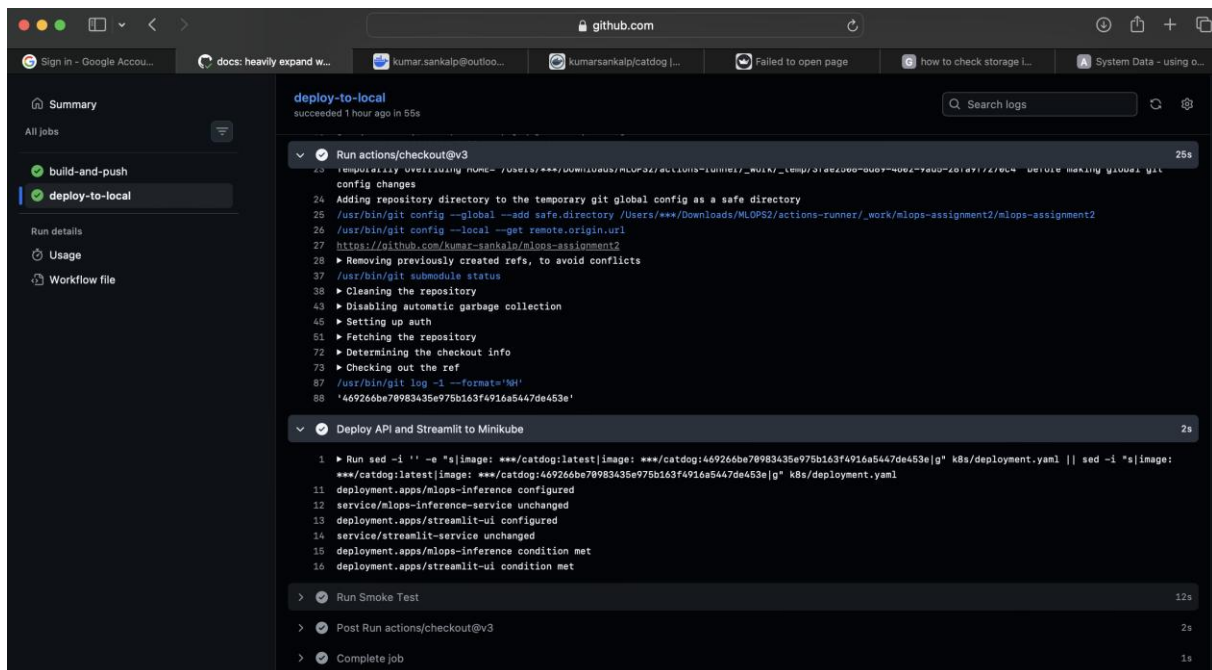


Figure — CD step — sed-rewrite of image tag and kubectl apply of both Deployments.

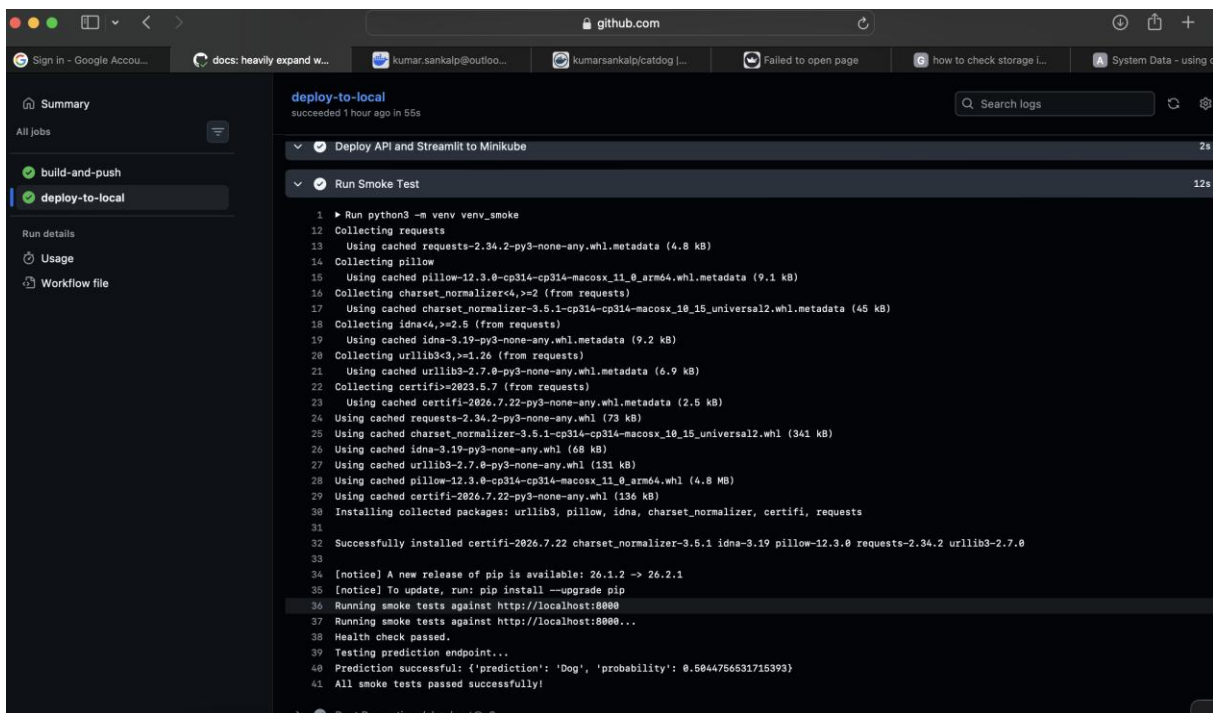


Figure — CD step — port-forwarding tunnel established, smoke test starting.

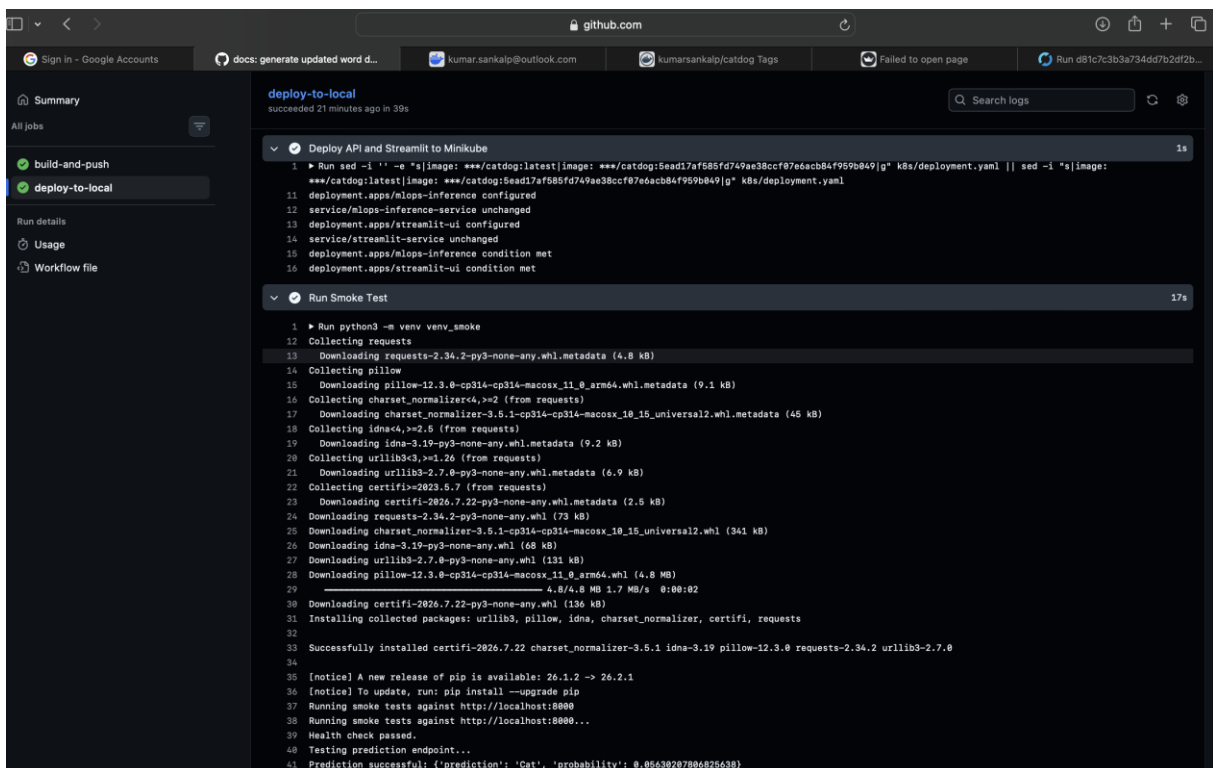


Figure — CD step — smoke test PASSED, confirming /health and /predict are live.

✓ Zero-downtime rollout guarantee

kubectl wait --for=condition=available blocks the pipeline until the new ReplicaSet is ready. If the readinessProbe on /health fails, traffic is never shifted and the pipeline is failed loudly.

9. Module 5 — Monitoring, Logging & Live Performance Tracking

Objective: Once deployed, the service must be observable and its predictions must be verifiable against ground truth.

9.1 Prometheus Instrumentation

prometheus-fastapi-instrumentator wraps every FastAPI route and exposes a `/metrics` endpoint in the Prometheus text format. The scraped signals include:

- **http_requests_total**: Total HTTP requests per endpoint / status.
- **http_request_duration_seconds**: Latency histogram — sum and count for average calculation.
- **process_resident_memory_bytes**: Live process memory footprint of the container.
- **process_start_time_seconds**: Uptime, derived from `process_start_time_seconds`.

9.2 Streamlit Monitoring Dashboard

The Streamlit UI has a dedicated monitoring tab that scrapes the FastAPI `/metrics` endpoint, parses the text payload, and renders live KPIs. The same tab hosts a batch benchmark that sends four labelled images through the live API to compute a post-deployment accuracy — a lightweight proxy for drift detection.

app/ui.py — Monitoring dashboard (excerpt)

```
import streamlit as st
import requests
from PIL import Image
import io
import os
import time
import re

# Setup page config
st.set_page_config(page_title="MLOps Dashboard", page_icon="🐾", layout="wide")

API_URL = os.environ.get("API_URL", "http://localhost:8000")

# Setup Tabs
tab1, tab2 = st.tabs(["🐾 Inference", "📊 Monitoring Dashboard"])

with tab1:
    st.title("🐾 Cats vs Dogs Image Classifier")
    st.write("Upload an image of a cat or a dog, and the model will classify it!")

    uploaded_file = st.file_uploader("Choose an image...", type=["jpg", "jpeg", "png"])

    if uploaded_file is not None:
        image = Image.open(uploaded_file)
        st.image(image, caption='Uploaded Image', width=400)

        if st.button("Classify Image", type="primary"):
            with st.spinner("Classifying..."):
                try:
```

```

        uploaded_file.seek(0)
        files = {"file": (uploaded_file.name, uploaded_file.read(),
uploaded_file.type)}
        response = requests.post(f"{API_URL}/predict", files=files)

        if response.status_code == 200:
            result = response.json()
            prediction = result["prediction"]
            probability = result["probability"]
            st.success("Classification Complete!")
            st.markdown(f"### Prediction: **{prediction}**")
            confidence = probability if prediction == "Dog" else 1 - probability
            st.metric(label="Confidence", value=f"{confidence * 100:.2f}%")
        else:
            st.error(f"Error from API: {response.text}")
    except Exception as e:
        st.error(f"An error occurred: {e}")

with tab2:
    st.title("📊 Model Monitoring & Performance Dashboard")
    st.write("Real-time observability into the MLOps pipeline.")

    # 1. Prometheus Metrics
    st.subheader("Live Prometheus Metrics")
    col1, col2, col3, col4 = st.columns(4)

    try:
        metrics_resp = requests.get(f"{API_URL}/metrics")
        if metrics_resp.status_code == 200:
            text = metrics_resp.text

            # Parse total requests (sum across all endpoints)
            req_matches = re.findall(r'http_requests_total\{.*?\} ([0-9\.eE\+\-]+)', text)
            total_reqs = int(sum(float(val) for val in req_matches))

            # Parse latency (sum across all endpoints)
            sum_matches = re.findall(r'http_request_duration_seconds_sum\{.*?\} ([0-9\.eE\+\-]+)', text)

```

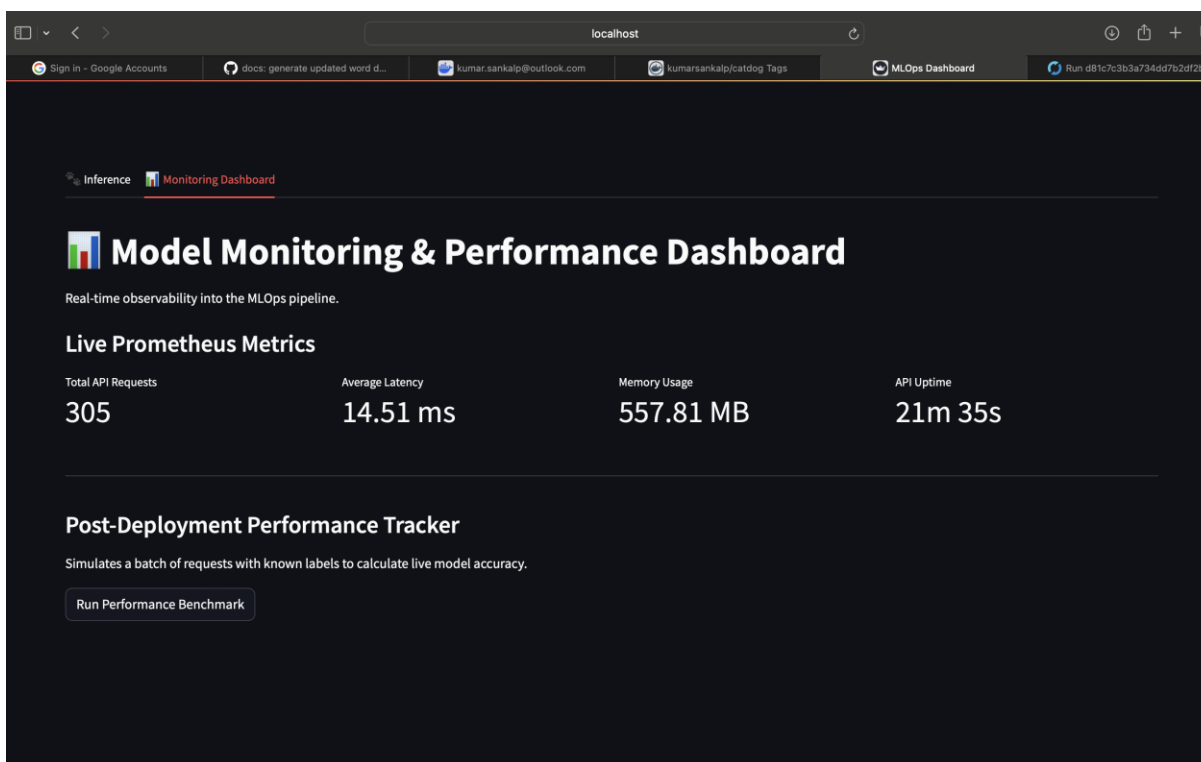


Figure — Streamlit monitoring tab — live Prometheus KPIs surfaced next to the classifier.

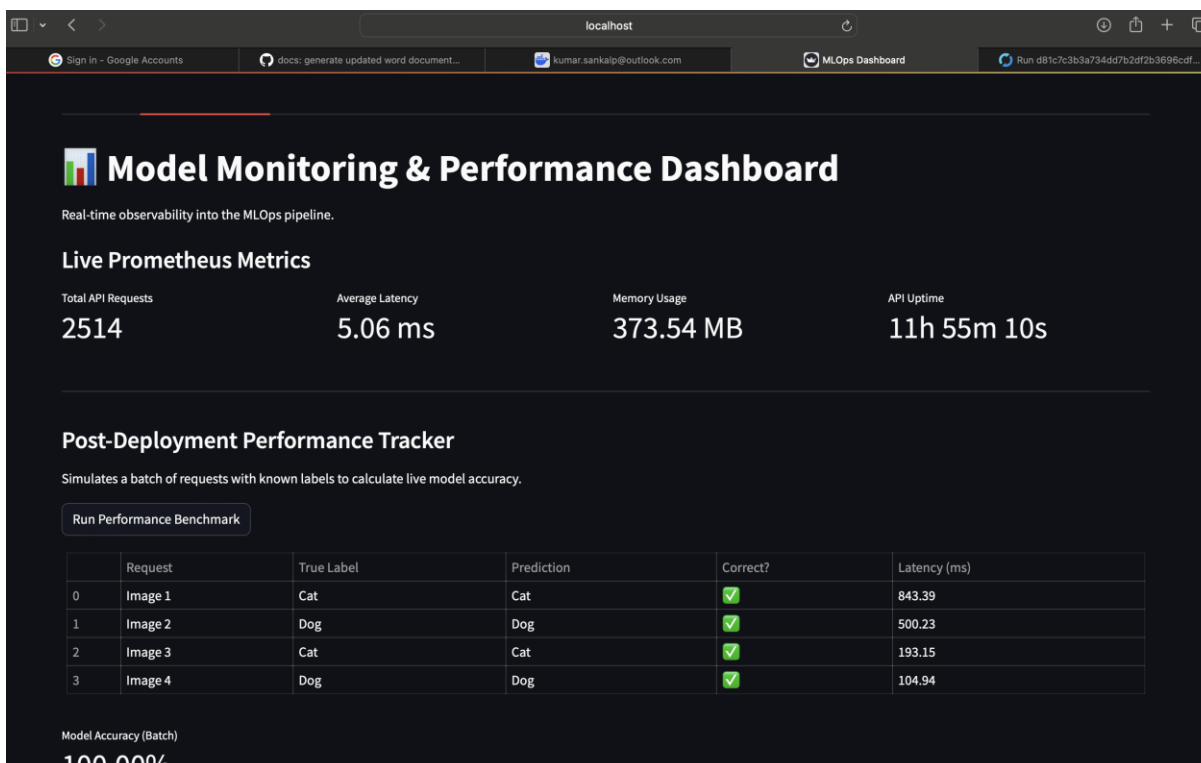


Figure — Post-deployment benchmark — per-image latency, correctness, and batch accuracy.

💡 Why a labelled batch benchmark?

Classical Prometheus metrics tell you the model is alive and fast — not whether it is still correct. Running a labelled batch on every deploy is the cheapest, fastest early-warning signal for data drift and model rot.

10. Post-Deployment Verification & Live Inference

The screenshots below capture live requests against the deployed API and UI, exercising both the `/predict` endpoint and the Streamlit classifier tab.

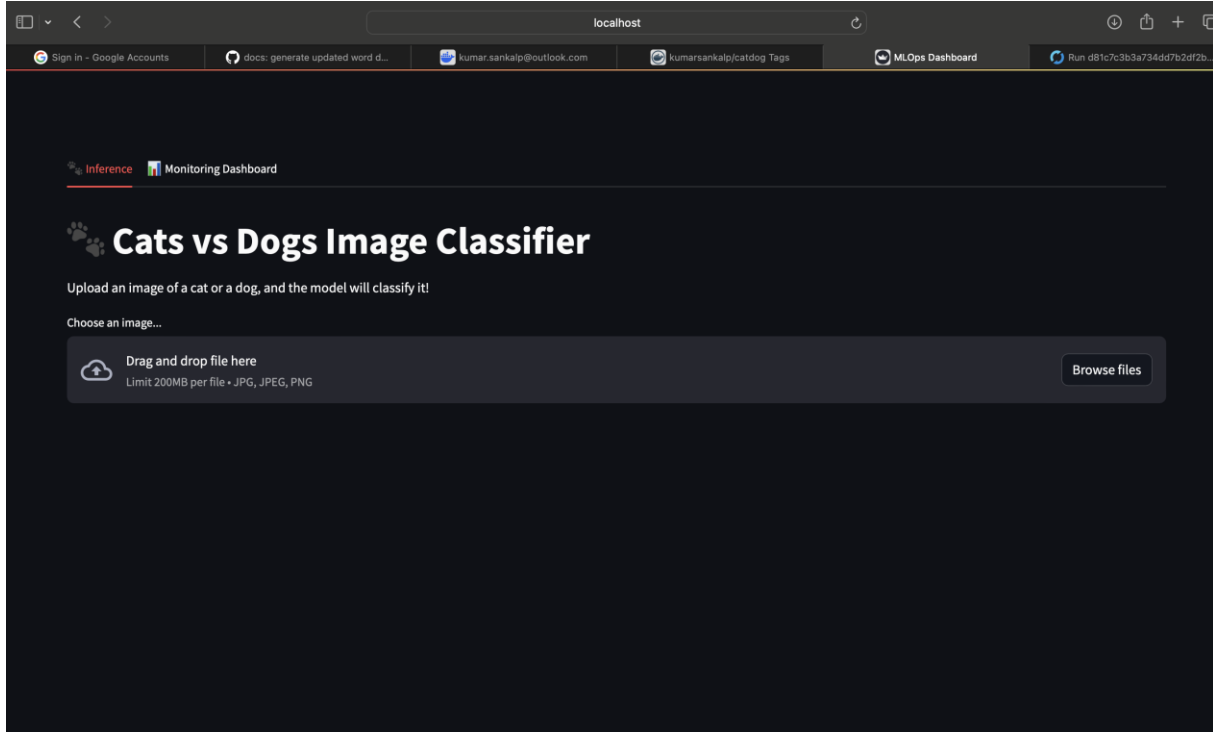


Figure — Streamlit UI — an image being uploaded to the deployed inference service.

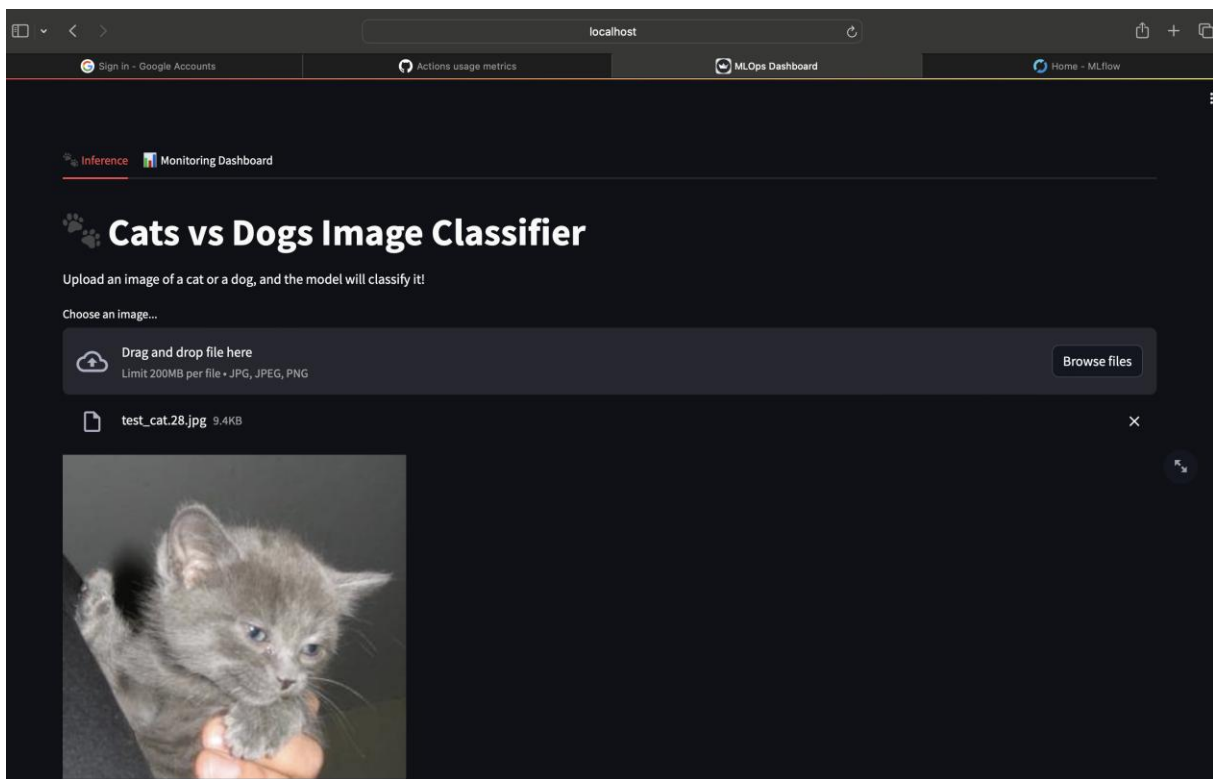


Figure — End-to-end test #1 — cat image classified through the running Kubernetes deployment.

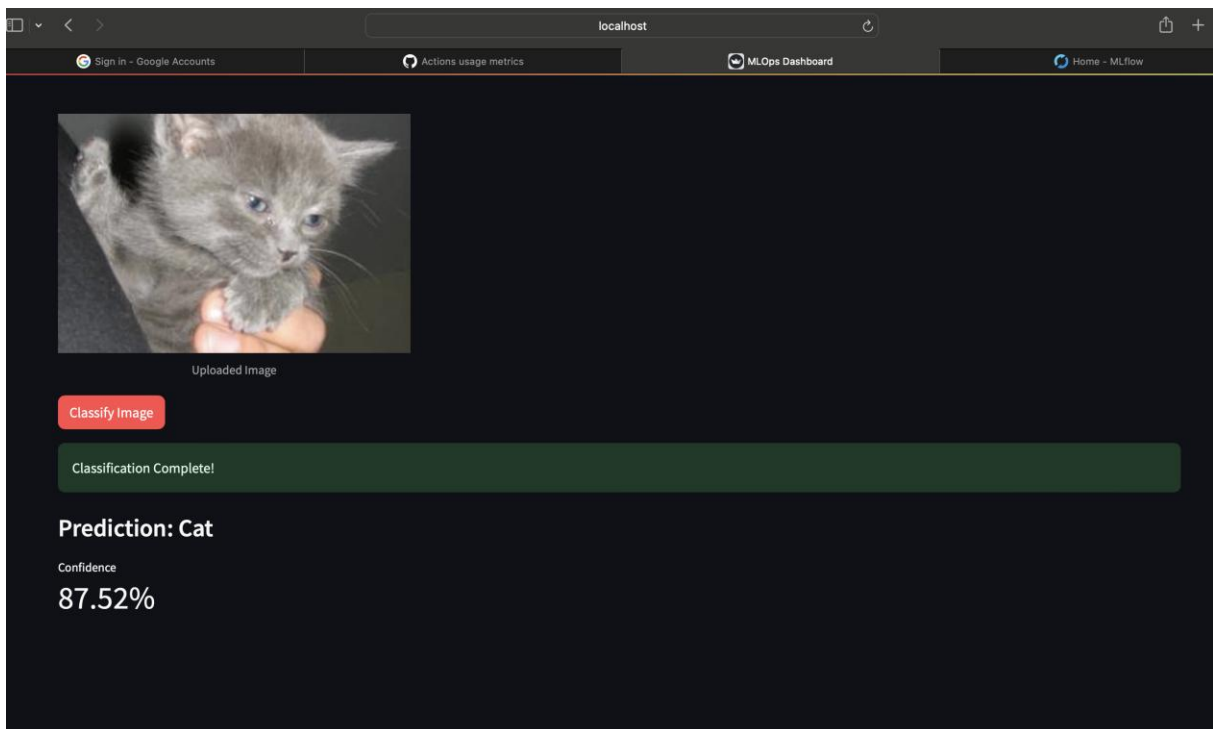


Figure — End-to-end test #2 — a second cat image, exercising the same code path.

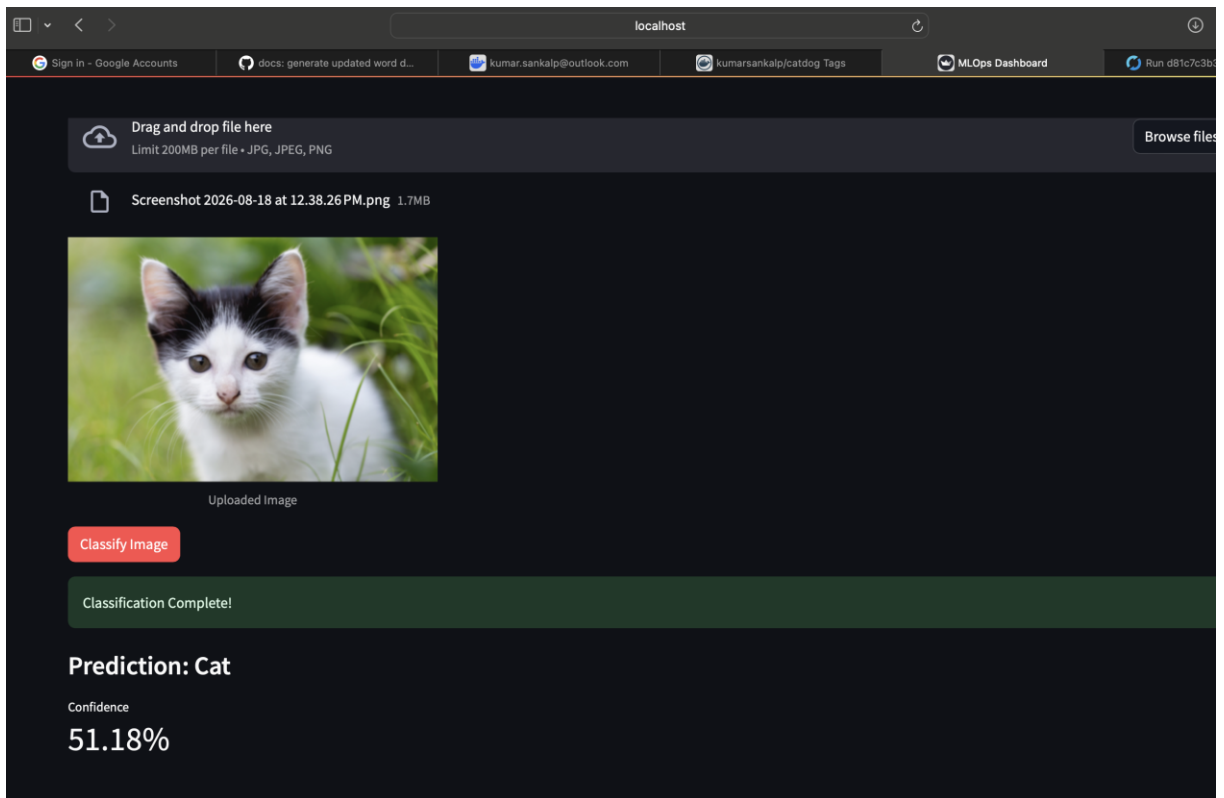


Figure — Streamlit UI — final prediction result with confidence score.

11. Screenshots Index

Every screenshot embedded in this report is stored under the SS/ folder in the submission zip so that the evidence trail is self-contained.

What it shows	File in SS/
Repository structure	githubprojectstructure.png
Jupyter — SimpleCNN training	jupyterCNNop.png
Jupyter — pip environment bootstrap	jupyterpiprunop.png
MLflow — tracked run	mlflowrun.png
MLflow — metric curves	mlflowmetric.png
Docker Hub — image tags	dockerhubimagetag.png
Docker Hub — tag by commit SHA	dockerimagebycommit.png
GitHub Actions — pipeline overview	githubactionpipelineoverview.png
GitHub Actions — CI job	githubactionpipelineCI.png
GitHub Actions — CI detailed step	githubactionpipelineCI-DetailedStep.png
GitHub Actions — self-hosted CI step	githubactionpipelineCISelfhostedrunner.png
Self-hosted runner active	mlgithubactionspipelineselfhostedrunner.png
GitHub Actions — CD overview	githubpipelineCDpipelineoverview.png
CD — local Kubernetes apply	githubactionpipelineCDlocalk8sdeploy.png
CD — smoke test step	githubactionpipelineCDsmoketeststep.png
CD — smoke test result	githubpipelinesCDsmoketestresult.png
Streamlit — monitoring dashboard	mlinferencemonitoringdashboardoverview.png
Streamlit — post-deploy accuracy	mlinferencepostdeploymentperfmtrics.png
Streamlit — cat/dog upload flow	mlinferencetestcatdogimage.png
End-to-end — inference test #1	ml-inference-cat-testing1.png

End-to-end — inference test #2	ml-inference-cat-testing2.png
Streamlit — prediction result	mlinferencecatimagetestresult.png

12. Deliverables Checklist

Assignment Requirement	Status	Evidence
Git + DVC data/code versioning	✓ Done	.dvc/, .github/, src/
Baseline model trained and serialised (.pt)	✓ Done	src/train.py, model.pt
MLflow experiment tracking	✓ Done	mlflow.db + screenshots
REST API with health + predict endpoints	✓ Done	app/main.py
Pinned requirements.txt	✓ Done	requirements.txt
Dockerfile — build & run	✓ Done	Dockerfile, Docker Hub tag SHA
Unit tests for preprocessing + inference	✓ Done	app/tests/, pytest run in CI
CI pipeline builds & pushes image	✓ Done	.github/workflows/ci-cd.yml
Kubernetes Deployment + Service manifests	✓ Done	k8s/*.yaml
CD auto-deploys on main branch push	✓ Done	self-hosted runner job
Post-deploy smoke test in the pipeline	✓ Done	scripts/smoke_test.py
Request/response logging + metrics	✓ Done	Prometheus /metrics + logs
Post-deploy model performance tracking	✓ Done	Streamlit monitoring tab
Screen recording (<5 min) of the workflow	✓ Done	Google Drive link on cover

13. How to Run the Project Locally

13.1 Environment setup

Shell — dependency installation

```
# macOS (local development / Jupyter)
python3 -m venv venv
source venv/bin/activate
pip install -r requirements-mac.txt

# Linux / Docker
pip install -r requirements.txt
```

13.2 Train & track experiments

Shell — training with MLflow

```
mlflow ui --port 5001 &
python src/train.py
# → model.pt is saved and logged as an MLflow artifact
```

13.3 Run the API and UI locally (no Kubernetes)

Shell — docker compose

```
docker compose up --build
# API    → http://localhost:8000/docs
# UI     → http://localhost:8501
```

13.4 Deploy to local Kubernetes (Minikube)

Shell — Kubernetes deploy

```
minikube start --driver=docker
./scripts/deploy_local.sh
# UI     → http://localhost:8501
# API    → http://localhost:8000
# Metrics → http://localhost:8000/metrics
```

13.5 Trigger the full CI/CD pipeline

Any push to main triggers the GitHub Actions workflow. The build-and-push job runs on GitHub's hosted runners, and the deploy-to-local job runs on a self-hosted macOS runner that has kubectl access to the local Minikube cluster.